

Кейс 1 (Неделя 1). Индивидуальное задание

Консольная утилита на Node.js для работы с внешним REST API

Формат выполнения: индивидуально. Ориентировочное время — 6 часов. Результат сдаётся в виде репозитория на GitHub / GitLab.

Проверяемые темы недели: асинхронный JavaScript (промисы, `async/await`, обработка ошибок); `fetch`, REST API и формат JSON; веб-протоколы HTTP/HTTPS (методы, статусы), тестирование в Postman; Git (ветки, `merge`, `Pull Request`, `code review`); Node.js (runtime, модули, `npm`, работа с файлами); переменные окружения и Docker — в дополнительной части.

1. Условие

Требуется написать консольную утилиту «Погодный дайджест» на Node.js. Утилита принимает название города (или список городов), получает по нему прогноз погоды из открытого REST API, выводит результат в читаемом виде в терминал и сохраняет отчёт в файл. При любых сбоях — неверные аргументы, неизвестный город, недоступность сети, ошибка на стороне API — утилита должна завершаться корректно и сообщать пользователю понятную причину.

Используется открытый API Open-Meteo, ключ доступа не требуется:

1. Геокодирование — получение координат по названию города:

GET

```
https://geocoding-api.open-meteo.com/v1/search?name={город}&count=1&language=ru&format=json
```

2. Прогноз погоды по координатам:

GET

```
https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&daily=temperature_2m_max,temperature_2m_min,precipitation_sum&forecast_days={n}&timezone=auto
```

По согласованию с наставником допускается другой открытый API (курсы валют, GitHub API и т. п.) при условии, что сохраняется цепочка из двух связанных запросов и выполняются все требования ниже.

2. Функциональные требования

1. Запуск утилиты выполняется командой вида:

```
node src/index.js --city "Нижний Новгород" --days 3
```

2. Параметр `--city` обязателен и поддерживает несколько городов через запятую. Параметр `--days` необязателен, по умолчанию 3, допустимый диапазон 1–7.
3. Для каждого города сначала выполняется запрос геокодирования, затем — запрос прогноза по полученным координатам.
4. Города обрабатываются параллельно. Ошибка по одному городу не должна прерывать обработку остальных.
5. В консоль выводится: название города, страна, координаты и таблица по дням — дата, минимальная и максимальная температура, суммарные осадки.
6. Отчёт сохраняется в файл `reports/{город}-{ГГГГ-ММ-ДД}.json`.

7. Кэширование: если отчёт по городу за текущую дату уже сохранён, данные берутся из файла без обращения к сети. Флаг `--no-cache` принудительно выполняет запрос заново.
8. Обрабатываются как минимум следующие ситуации: отсутствующий или некорректный аргумент; город не найден (пустой результат геокодинга); ответ со статусом 4xx; ответ со статусом 5xx; отсутствие сети; превышение таймаута; некорректный JSON в ответе.
9. Код завершения процесса: 0 — успех, 1 — ошибка.

3. Технические требования

- Node.js версии 20 и выше, встроенный `fetch`. Сторонние HTTP-клиенты (`axios` и аналоги) не используются.
- Весь асинхронный код — на `async/await`, единый стиль по проекту.
- Ограничение времени запроса реализуется через `AbortController`; значение таймаута берётся из переменной окружения (по умолчанию 5000 мс).
- Проект разбит на модули по зонам ответственности, например: `src/index.js` (разбор аргументов и запуск), `src/api/` (клиент внешнего API), `src/services/` (бизнес-логика), `src/storage/` (файлы и кэш), `src/format/` (вывод).
- Используется одна модульная система — ESM (`"type": "module"`) или CommonJS, без смешения.
- В `package.json` описаны зависимости и `npm`-скрипты запуска.
- Настраиваемые параметры (базовые URL, таймаут, каталог отчётов, единицы измерения) выносятся в переменные окружения. В репозиторий добавляется `.env.example`, файл `.env` добавляется в `.gitignore`.
- Каталоги `node_modules` и `reports` в репозиторий не попадают.

4. Требования к оформлению в Git

- Создан репозиторий на GitHub / GitLab.
- Работа ведётся минимум в трёх отдельных ветках по задачам (например: `feat/cli`, `feat/api-client`, `feat/error-handling`). Прямых коммитов в `main` нет.
- Коммиты атомарные, сообщения осмысленные и в едином стиле (рекомендуется Conventional Commits: `feat`, `fix`, `refactor`, `docs`).
- Изменения вливаются в `main` через Pull Request с описанием: что сделано и как проверить.
- Pull Request проходит code review наставника. Все замечания отработаны: внесены правки или дан аргументированный ответ в комментариях. После этого PR вливается.

5. Тестирование API в Postman

- Создана коллекция с обоими используемыми запросами.
- Значения вынесены в переменные коллекции или окружения (`{{baseUrl}}`, `{{city}}`, `{{lat}}`, `{{lon}}`).
- Сохранены примеры ответов: успешный, «город не найден» и ответ с ошибкой (например, при некорректных параметрах запроса) — с указанием кодов статусов.

- Коллекция экспортирована в JSON и добавлена в репозиторий, например в docs/postman/.

6. Требования к README

README.md должен содержать: краткое описание утилиты, требования к окружению, инструкцию по установке, перечень переменных окружения, команды запуска со всеми параметрами, пример вывода в консоль, описание обрабатываемых ошибок и кодов завершения, краткое описание структуры проекта.

7. Дополнительная часть (по желанию)

Выполняется сверх основного задания и даёт бонусные баллы.

- Dockerfile: образ собирается, утилита запускается в контейнере, параметры передаются через переменные окружения.
- Настроены ESLint и Prettier, проверка запускается npm-скриптом и проходит без ошибок.
- Простая HTML-страница, которая читает сохранённый JSON-отчёт и выводит его на страницу средствами DOM API.
- Базовые тесты на разбор ответа API и обработку ошибочных сценариев.

8. Что сдаётся

1. Ссылка на репозиторий.
2. Ссылка на влитый Pull Request с пройденным code review.
3. Экспортированная коллекция Postman в составе репозитория.
4. Устная защита 5–10 минут: демонстрация работы утилиты и ответы на вопросы по коду.

9. Критерии оценивания

Каждый критерий оценивается по шкале от 0 до 3 баллов:

Балл	Значение
0	Критерий не выполнен
1	Критерий выполнен частично
2	Критерий выполнен полностью
3	Критерий перевыполнен (решение лучше требуемого)

Итоговый результат = сумма по всем критериям (балл × вес). Максимум по обязательным критериям — 84 балла (28 × 3). Бонусные критерии добавляют до 6 баллов сверх этого.

Критерий	Вес
Функциональность и работоспособность	
Утилита устанавливается и запускается по инструкции из README, основной сценарий отрабатывает без падений	2

Критерий	Вес
Реализован CLI-интерфейс: параметры --city и --days, значение по умолчанию, валидация введенных аргументов	1
Данные корректно получены из внешнего REST API, вывод соответствует запрошенному городу и количеству дней	2
Отчёт сохраняется в файл, повторный запуск за тот же день использует кэш вместо сетевого запроса	1
Асинхронный JavaScript	
Асинхронный код написан на async/await в едином стиле, без вложенных колбэков и смешения с цепочками .then	1
Независимые запросы (несколько городов) выполняются параллельно через Promise.all / Promise.allSettled, сбой по одному городу не прерывает остальные	1
Реализовано ограничение времени запроса (AbortController или аналог), таймаут настраивается	0,5
Работа с HTTP и REST API	
Запросы выполняются через fetch с корректным методом и заголовками, URL и query-параметры собираются через URL / URLSearchParams (без ручной конкатенации строк)	1
Проверяется HTTP-статус ответа, реакция на 4xx и 5xx различается и описана	1
JSON корректно разбирается, извлекаются только нужные поля, результат выводится в читаемом человеку виде	1
Обработка ошибок	
В проекте отсутствуют необработанные исключения и unhandled promise rejection: сетевые сбои, некорректный JSON, неизвестный город и ошибки API перехватываются	2
Ошибки выводятся пользователю понятным сообщением (без стек-трейса), процесс завершается с ненулевым кодом выхода	1
Node.js, модули и инструменты	
Код разбит на модули по зонам ответственности (CLI, клиент API, форматирование, хранение), используется одна модульная система	1
package.json оформлен корректно: npm-скрипты запуска, актуальные зависимости; node_modules и артефакты добавлены в .gitignore	1
Работа с файловой системой выполняется асинхронно (fs/promises), пути формируются через модуль path	0,5
Конфигурация вынесена в переменные окружения, в репозитории есть .env.example, сам .env игнорируется, хардкода параметров в коде нет	0,5
Git	
История коммитов осмысленная: атомарные коммиты, понятные сообщения в едином стиле	1
Работа велась в отдельных ветках по задачам, прямых коммитов в main нет	1
Оформлен Pull Request с описанием сути изменений и способа проверки	2
Тестирование API и документация	

Критерий	Вес
Коллекция Postman со всеми используемыми запросами экспортирована и добавлена в репозиторий	1
Сохранены примеры успешного и ошибочного ответов с указанием кодов статусов	0,5
README содержит описание, установку, переменные окружения, запуск с параметрами, пример вывода и перечень обрабатываемых ошибок	1
Качество кода и защита решения	
Код читаемый: единый стиль форматирования, осмысленные имена, отсутствует закомментированный и мёртвый код, отладочный вывод убран	1
Участник объясняет архитектуру решения и принцип работы любой его части, отвечает на вопросы по своему коду	2
Дополнительные задания (бонус)	
Написан Dockerfile, образ собирается, утилита запускается в контейнере	0,5
Настроен линтер/форматтер (ESLint, Prettier), команда проверки проходит без ошибок	0,5
Сделана HTML-страница, отображающая сохранённый отчёт средствами DOM API	0,5
Написаны базовые тесты на разбор ответа API и обработку ошибок	0,5
Сумма весов: обязательные критерии	28
Сумма весов: бонусные критерии	2

10. Шкала итоговой оценки

Взвешенная сумма баллов	Итоговая оценка
71,5 и выше (85 % и более)	Отлично
от 59,0 до 71,0 (70–84 %)	Хорошо
от 42,0 до 58,5 (50–69 %)	Удовлетворительно
менее 42,0 (менее 50 %)	Не зачтено, требуется доработка

Процент считается от максимума по обязательным критериям. Бонусные баллы прибавляются к итоговой сумме и могут повысить оценку.